

API Access

CRKN provides programmatic access to discovery metadata through Canadiana Collections and Infrastructure APIs. Combine these endpoints with search parameters, facets, paging, and sorting.

ON THIS PAGE

Catalog API endpoints

Catalog query parameters

Catalog response structure

Catalog examples

IIIF overview

IIIF Image API

IIIF Presentation API (Manifest)

IIIF Content Search API

Catalog API endpoints

Use these catalog endpoints for search, records, and facets. The `format` query parameter is the most reliable option when IDs include dots (for example `oocihm.N_00591`).

Catalog API endpoints

ENDPOINT	METHOD AND PATH
Search endpoint	GET <code>/catalog?search_field=all_fields&q=&format=json</code>
Single record endpoint	GET <code>/catalog/:id?format=json</code>
Facet endpoint	GET <code>/catalog/facet/:facet_id?search_field=all_fields&q=&format=json</code>
Single record export endpoint	GET <code>/catalog/:id?format=marc marcxml xml</code>

Catalog query parameters

Catalog query parameters

PARAMETER	TYPE	DESCRIPTION	VALUES / DEFAULT
q	string	Search query text.	Default: empty string
search_field	enum	Search field to run the query against.	all_fields, full_title_tsim, author_tsim, subject_tsim, tx_gen
lang	enum	Response locale.	en, fr
page	integer	Result page number (starts at 1).	Default: 1
per_page	integer	Number of results per page.	10, 20, 50, 100
sort	enum	Sort key for result ordering.	relevance, year-desc, year-asc, date-added-desc, date-added-asc
f[field_name][]=value	array	Facet filters to constrain results.	pub_date_ssim, language_ssim_str, depositor_tsim_str, subject_ssim_str, author_ssim_str, collectionen_path, collectionfr_path, serial_title_str, is_issue_str, is_serial_str. Example: f[language_ssim_str][]=English
range[pub_date_ssim][begin] range[pub_date_ssim][end]	integer	Publication-year range filter.	Example: 1900 to 1910

format	enum	Output format for the response.	Search: <code>html</code> , <code>json</code> ; Single record: <code>html</code> , <code>json</code> , <code>marc</code> , <code>marcxml</code> , <code>xml</code>
---------------	------	---------------------------------	--

Catalog response structure

A search request such as `/catalog?search_field=all_fields&q=&format=json` returns:

Catalog response structure fields

FIELD	DESCRIPTION
links	Navigation URLs (<code>self</code> , <code>next</code> , <code>last</code>).
meta.pages	Page state and counts (<code>current_page</code> , <code>total_pages</code> , <code>limit_value</code> , <code>total_count</code>).
data[]	Search result records with <code>id</code> , <code>type</code> , <code>attributes</code> , and <code>links.self</code> .
included[]	Reusable API objects for facet definitions, search fields, and sort options.

```
ResponseJSON{
  "links": {"self": "...", "next": "...", "last": "..."},
  "meta": {
    "pages": {
      "current_page": 1,
      "total_pages": 246,
      "limit_value": 10,
      "total_count": 2454
    }
  },
  "data": [
    {
      "id": "oocihm.N_00591",
      "type": "Newspaper",
      "attributes": {
        "title": "Keneder adler",
        "published_ssm": {
          "type": "document_value",
          "attributes": {"label": "Published", "value":
```

```

"..."}
        },
        "language_ssim": {
            "type": "document_value",
            "attributes": {"label": "Language", "value":
"..."}
        }
    },
    "links": {"self":
"http://localhost:3000/catalog/oocihm.N\_00591?lang=en"}
    }
],
"included": [
    {"type": "facet", "id": "language_ssim_str"},
    {"type": "search_field", "id": "all_fields"},
    {"type": "sort", "id": "date-added-desc"}
]
}

```

Many `attributes.*.attributes.value` values are HTML fragments for display. If you are building a pure data pipeline, plan to strip or sanitize markup.

Catalog examples

Example 1: Default catalog JSON response

Fetch the default catalog payload and read total result count.

Code`python``import requests`

```

BASE_URL = "http://localhost:3000"

response = requests.get(
    f"{BASE_URL}/catalog",
    params={"search_field": "all_fields", "q": "", "format":
"json"},
    timeout=30,
)
response.raise_for_status()
catalog = response.json()
print(catalog["meta"]["pages"]["total_count"])

```

Example 2: Query with sorting and pagination

Run a keyword query and control order, page size, and page number.

Code

```
pythonimport requests
```

```
BASE_URL = "http://localhost:3000"
```

```
response = requests.get(
    f"{BASE_URL}/catalog",
    params={
        "q": "maps",
        "search_field": "all_fields",
        "sort": "date-added-desc",
        "per_page": 20,
        "page": 1,
        "format": "json",
    },
    timeout=30,
)
response.raise_for_status()
maps = response.json()
```

Example 3: Facet filters

Apply facet constraints to narrow results by language and collection path.

Code

```
pythonimport requests
```

```
BASE_URL = "http://localhost:3000"
```

```
english_response = requests.get(
    f"{BASE_URL}/catalog",
    params={
        "q": "",
        "search_field": "all_fields",
        "f[language_ssim_str][]": "English",
        "format": "json",
    },
    timeout=30,
)
english_response.raise_for_status()
english_only = english_response.json()
```

```

newspapers_response = requests.get(
    f"{BASE_URL}/catalog",
    params={
        "q": "",
        "search_field": "all_fields",
        "f[collectionen_path][]": "Serials/Newspapers",
        "format": "json",
    },
    timeout=30,
)
newspapers_response.raise_for_status()
newspapers = newspapers_response.json()

```

Example 4: Publication year range

Filter records by year interval using the configured range facet field.

Codepythonimport requests

```
BASE_URL = "http://localhost:3000"
```

```

response = requests.get(
    f"{BASE_URL}/catalog",
    params={
        "q": "",
        "search_field": "all_fields",
        "range[pub_date_ssim][begin]": 1900,
        "range[pub_date_ssim][end]": 1910,
        "format": "json",
    },
    timeout=30,
)
response.raise_for_status()
year_range = response.json()

```

Example 5: Single record in multiple formats

Retrieve the same record as JSON, MARC, and MARCXML from one endpoint.

Codepythonimport requests

```
BASE_URL = "http://localhost:3000"
```

```
json_response = requests.get(
```

```
f"{BASE_URL}/catalog/oocihm.N_00591",
params={"format": "json"},
timeout=30,
)
json_response.raise_for_status()
record_json = json_response.json()

marc_response = requests.get(
    f"{BASE_URL}/catalog/oocihm.N_00591",
    params={"format": "marc"},
    timeout=30,
)
marc_response.raise_for_status()
record_marc = marc_response.text

marcxml_response = requests.get(
    f"{BASE_URL}/catalog/oocihm.N_00591",
    params={"format": "marcxml"},
    timeout=30,
)
marcxml_response.raise_for_status()
record_marcxml = marcxml_response.text
```

IIIF API

IIIF API endpoints

IIIF is a shared framework for publishing, linking, and reusing digital cultural heritage objects. IIIF resources are published as Linked Open Data using [JSON-LD](#), which is a JSON serialization of [RDF](#). This lets clients follow stable identifiers across manifests, canvases, annotations, and image services.

Endpoints are grouped in the Image, Manifest, and Content Search sections below.

Official IIIF API docs: iiif.io/api.

IIIF Image API

IIIF Image API endpoints

ENDPOINT	PATH
Image info endpoint	GET {image_service}/info.json
Image render endpoint	GET {image_service}/{region}/{size}/{rotation}/{quality}.{format}

Full specification: [IIIF Image API 2.1](#).

Response formats

IIIF Image API response formats

ENDPOINT	RESPONSE FORMAT
Image info endpoint (info.json)	JSON (IIIF Image API)
Image render endpoint	Binary image bytes (for example image/jpeg)

IIIF Image API request parameters

PARAMETER	TYPE	DESCRIPTION	VALUES / EXAMPLES
region	string	Image region to extract.	full, x,y,w,h, pct:x,y,w,h
size	string	Target output dimensions.	full, w,,h, w,h, pct:n
rotation	number	Output rotation angle.	0, 90, 180, 270
quality	enum	Rendering quality profile supported by the service.	default, color, gray, bitonal
format	enum	Binary image format returned by the endpoint.	jpg

IIIF Image API response shape

info.json returns JSON metadata:

```
ResponseJSON{
  "@context": "http://iiif.io/api/image/2/context.json",
  "@id": "{image_service}",
  "protocol": "http://iiif.io/api/image",
```



```

    "width": 3936,
    "height": 3200,
    "sizes": [{"width": 123, "height": 100}, "..."],
    "tiles": [{"width": 512, "height": 512, "scaleFactors": [1, 2,
4, 8, 16, 32]}],
    "profile": [
        "http://iiif.io/api/image/2/level1.json",
        {
            "formats": ["jpg"],
            "qualities": ["bitonal", "default", "gray", "color"],
            "supports": ["regionByPx", "sizeByW", "rotationBy90s",
"..."]
        }
    ]
}

```

IIIF Image API examples

Example: IIIF image requests

Use the image service to inspect capabilities and request common derivatives.

Code`python`import requests

```

def iiif_image_url(
    service_id,
    region="full",
    size="full",
    rotation="0",
    quality="default",
    fmt="jpg",
):
    return
    f"{service_id}/{region}/{size}/{rotation}/{quality}.{fmt}"

service_id = "https://image-
tor.canadiana.ca/iiif/2/69429%2Fc2025000026xr85"

# Discover capabilities
info = requests.get(f"{service_id}/info.json", timeout=30).json()
print(info["width"], info["height"]) # 3936 3200

```

```

# Full image resized to width 500
img_small = requests.get(iiif_image_url(service_id, size="500,"),
    timeout=30)
print(img_small.headers["Content-Type"]) # image/jpeg

# Crop by percent region, then resize
img_crop = requests.get(
    iiif_image_url(service_id, region="pct:25,25,50,50",
size="800,"),
    timeout=30,
)

# Rotated output
img_rotated = requests.get(iiif_image_url(service_id, size="500,",
rotation="90"), timeout=30)

# Alternate quality
img_gray = requests.get(iiif_image_url(service_id, size="500,",
quality="gray"), timeout=30)

```

Example: Binary image response

Read a rendered image response and verify MIME type before using raw bytes.

Codepythonimport requests

```

response = requests.get(
    "https://image-
tor.canadiana.ca/iiif/2/69429%2Fc2025000026xr85/full/500,/0/default.
jpg",
    timeout=30,
)
response.raise_for_status()

print(response.headers["Content-Type"]) # image/jpeg
binary_jpeg = response.content          # bytes
print(len(binary_jpeg))

```

IIIF Presentation API (Manifest)

The example below is based on manifest [m2025000007606h](#) (collection [69429](#)). It is a IIIF Presentation 3 manifest with 842 canvases.

IIIF Manifest endpoint

ENDPOINT	PATH
Manifest endpoint	GET {presentation_service}/69429/m2025000007606h

Full specification: [IIIF Presentation API 3.0](#).

Response formats

IIIF Manifest response formats

ENDPOINT	RESPONSE FORMAT
Manifest endpoint	JSON (IIIF Presentation 3)

Manifest response shape (Presentation 3)

```
ResponseJSON{
  "@context": "http://iiif.io/api/presentation/3/context.json",
  "id": "{presentation_service}/69429/m2025000007606h",
  "type": "Manifest",
  "label": {"en": ["Census returns for the 1901 Canadian census"]},
  "metadata": [
    {
      "label": {"en": ["Slug"]},
      "value": {"en": ["oocihm.lac_reel_t6428"]}
    }
  ],
  "summary": {"en": ["..."]},
  "items": [
    {
      "id":
"{presentation_service}/canvas/69429/c2025000026xr85",
      "type": "Canvas",
      "width": 3936,
      "height": 3200,
      "label": {"en": ["Image 1"]},
      "items": [
```

```

{
  "type": "AnnotationPage",
  "items": [
    {
      "type": "Annotation",
      "motivation": "painting",
      "target":
        "{presentation_service}/canvas/69429/c2025000026xr85",
      "body": {
        "id":
          "{image_service}/full/max/0/default.jpg",
        "type": "Image",
        "format": "image/jpeg",
        "width": 3936,
        "height": 3200,
        "service": [
          {
            "id": "{image_service}",
            "type": "ImageService2",
            "profile": "level2"
          }
        ]
      }
    }
  ]
},
{
  "thumbnail": [{"id":
    "{image_service}/full/200,/0/default.jpg", "type": "Image",
    "format": "image/jpeg"}]
}
]
}

```

IIIF Content Search API

IIIF Content Search endpoint

ENDPOINT	PATH
Content Search	GET /search/{repository_id}/{manifest_id}?q={term}

endpoint (template)	
Content Search endpoint (example)	GET https://crkn-iiif-content-search.azurewebsites.net/search/69429/m0tt4fn16j1j?q=roi

Full specification: [IIIF Content Search API 2.0](#).

Response formats

IIIF Content Search response formats

ENDPOINT	RESPONSE FORMAT
Content Search endpoint	JSON-LD (IIIF Search API 2.0); top-level type is AnnotationPage , items contains hit annotations, and next is present when results are paginated.

Response shape (IIIF Search API 2.0)

IIIF Content Search response shape fields

FIELD	DESCRIPTION
@context	JSON-LD context (for example http://iiif.io/api/search/2/context.json).
id	Content Search endpoint URL for the manifest being searched.
type	AnnotationPage .
items[]	Array of hit annotations; each item includes motivation , body.value , and target with an #xywh= selector.
next	Optional pagination URL (for example ?start=100&q=roi).

```
ResponseJSON{
  "@context": "http://iiif.io/api/search/2/context.json",
  "id": "https://crkn-iiif-content-search.azurewebsites.net/search/69429/m0tt4fn16j1j",
  "type": "AnnotationPage",
  "items": [
    {
      "id": "
```

```

    api.azurewebsites.net/manifest/69429/m0tt4fn16j1j/canvas/69429/c05t3g16rb0q/text/at/667%2C253%2C100%2C110",
    "type": "Annotation",
    "motivation": "highlighting",
    "body": {
        "type": "TextualBody",
        "value": "Roi",
        "format": "text/plain"
    },
    "target": "https://crkn-iiif-api.azurewebsites.net/manifest/69429/m0tt4fn16j1j/canvas/69429/c05t3g16rb0q#xywh=667,253,100,110"
},
],
"next": "https://crkn-iiif-content-search.azurewebsites.net/search/69429/m0tt4fn16j1j?start=100&q=roi"
}

```

Example: ARK to manifest and content search

Extract ARK from catalog metadata, build IIIF path, fetch manifest, then run content search.

```

Codepythonimport re
import requests

```

```

BASE_URL = "http://localhost:3000"
PRESENTATION_SERVICE = "{presentation_service}"
CONTENT_SEARCH_SERVICE = "{content_search_service}"

```

1) Get a catalog record and extract ARK href from HTML value

```

record = requests.get(
    f"{BASE_URL}/catalog/oocihm.N_00591",
    params={"format": "json"},
    timeout=30,
).json()
ark_html =
record["data"]["attributes"]["ark"]["attributes"]["value"]
ark_url = re.search(r'href="([^\"]+)"', ark_html).group(1)

```

2) Normalize ARK path for IIIF services

```

ark_path = re.sub(r"^https?://n2t\\.net/ark:/?|^ark:/?", "",
ark_url).lstrip("/")

```

```

# "69429/m2025000007606h"

# 3) Fetch manifest
manifest = requests.get(f"{PRESENTATION_SERVICE}/{ark_path}",
    timeout=30).json()
print(manifest["type"], len(manifest["items"])) # Manifest 842

# 4) Derive image service from first canvas
image_service =
manifest["items"][0]["items"][0]["items"][0]["body"]["service"][0]["
id"]
info = requests.get(f"{image_service}/info.json", timeout=30).json()
print(info["width"], info["height"])

# 5) Run IIIF Content Search
search = requests.get(
    f"{CONTENT_SEARCH_SERVICE}/{ark_path}",
    params={"q": "canada"},
    timeout=30,
).json()
print(search["type"], len(search["items"]))

```

For a broader overview of IIIF concepts and tools, see [What is IIIF?](#).